

Stock-AI: AI Integrated Stock Trading Platform

Spring 2026

COM S 4020 - UG_HF_03

<https://seniord.cs.iastate.edu/2026-March-14/>

Client - Dr. Hung Phan

Instructor - Dr. Simanta Mitra

Teaching Assistant - Haniyeah Fekrimandi

Team Members:

Andrew Butler - Backend development, client communication

Matthew He - Full Stack development

Ethan Roe - Frontend development

Aaron Trelstad - Full-Stack development, AI integration

Problem

Stock trading is a difficult skill to learn. The stock market is constantly changing and evolving, and gaining intuition on smart investments is time consuming and difficult to test without making real investments. Learning platforms exist, but they are expensive services that add on to the already financially risky task of trading.

Solution

Our platform, Stock-AI, aims to help beginners learn stock trading completely for free with no risk. It accomplishes this in two ways: first, the Stock-AI platform doesn't charge users. There is no subscription or service cost to the user, and all trades are simulated to eliminate the risk of financial loss. Second, the platform includes AI integration to allow the user to ask questions and get summaries of relevant information, as well as to get recommendations on any given stock.

Requirements

Our team was initially tasked with identifying important features from existing platforms such as BunnyQuant and RobinHood. We created an initial proposed list of features which then guided our development of Stock-AI. We compiled and gave this list to Dr. Phan, who gave us the green light on them. While the final Stock-AI application has many of the same ideas, it is not a complete implementation of the initial featurelist; as a team, we continually gained domain knowledge and evolved what we included, given the approval of Dr. Phan on new features. The initial feature document includes user security, portfolios, mock trading, and stock history viewing, but notably does not include the leagues feature (described below). The original proposed featurelist document is appended at the end of this report.

Major Stock-AI elements

Stock-AI has 3 main pages that users interact with: the Dashboard, User Portfolios, and Leagues.

1. Dashboard:

The Dashboard serves as the primary way that users get information about the stock market. From here, they can look up any ticker, view its price history, see linked news articles about that company, as well as ask the AI assistant questions and get a recommendation for buy/sell/hold with associated reasoning. Trades can also be performed here, although they must be linked through a singular portfolio the user has. See **Figure 1** below for a view of the dashboard.

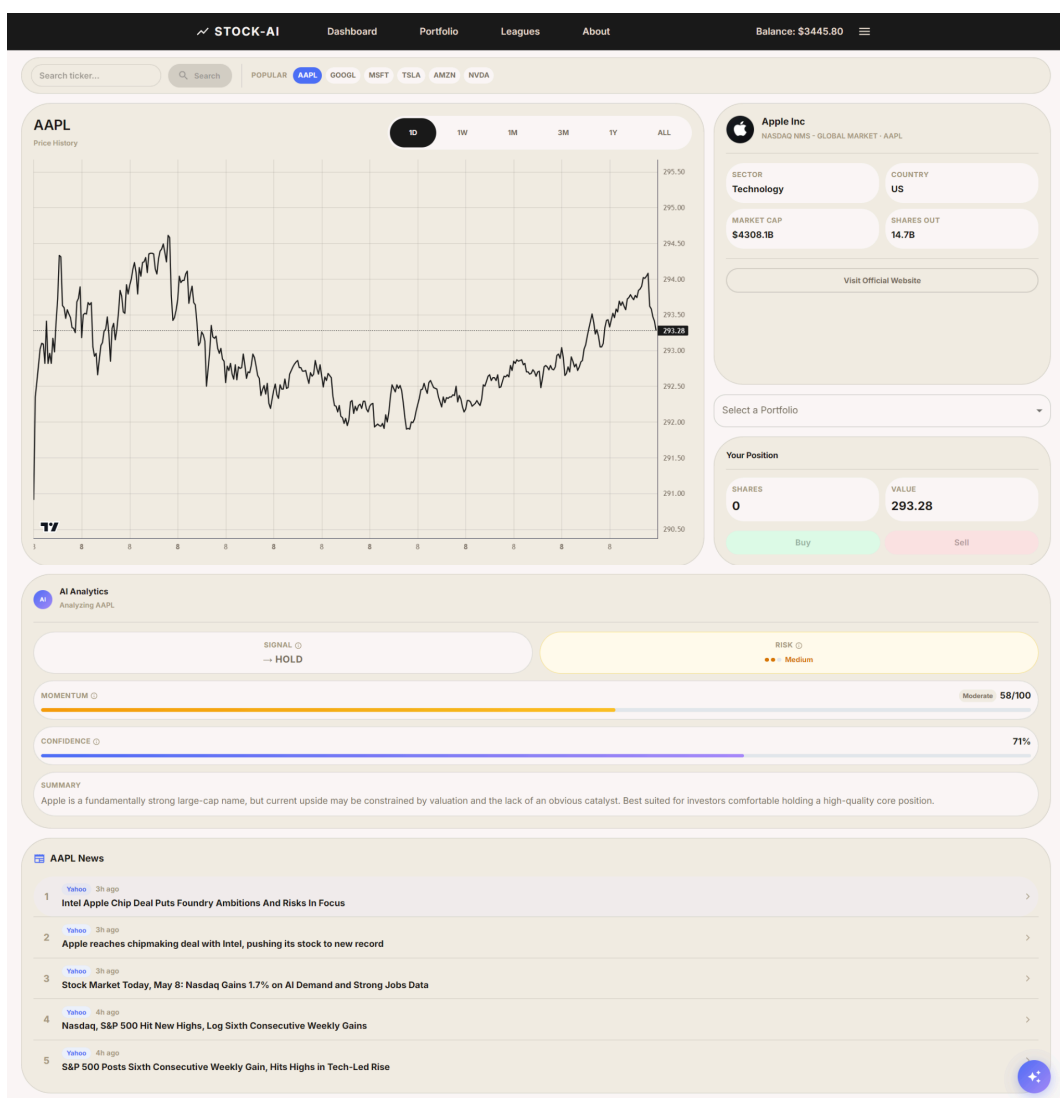


Figure 1: Dashboard page of Stock-AI platform. Note the price graph in the upper left, company information on the upper right, AI analytics in the middle, and relevant news at the bottom.

2. Portfolios:

The Portfolios view allows a user to create, manage, and view their portfolios, including any held positions they contain. Viewing positions gives their current price per share, the average cost basis of the user's owned shares, and their net gain/loss at the current market price. The user can choose to buy/sell from here as well, which will automatically lead them to the Dashboard with that given stock. **Figure 2** on the next page illustrates this view.

Portfolios also have a performance view, which enables a user to see their portfolio value over time. The graph view allows selection of time period as well as the inclusion or exclusion of any held positions. The performance view also shows trade history, stock allocation, and specific value numbers for the portfolio. See **Figure 3** on the next page for an example of the portfolio performance view.

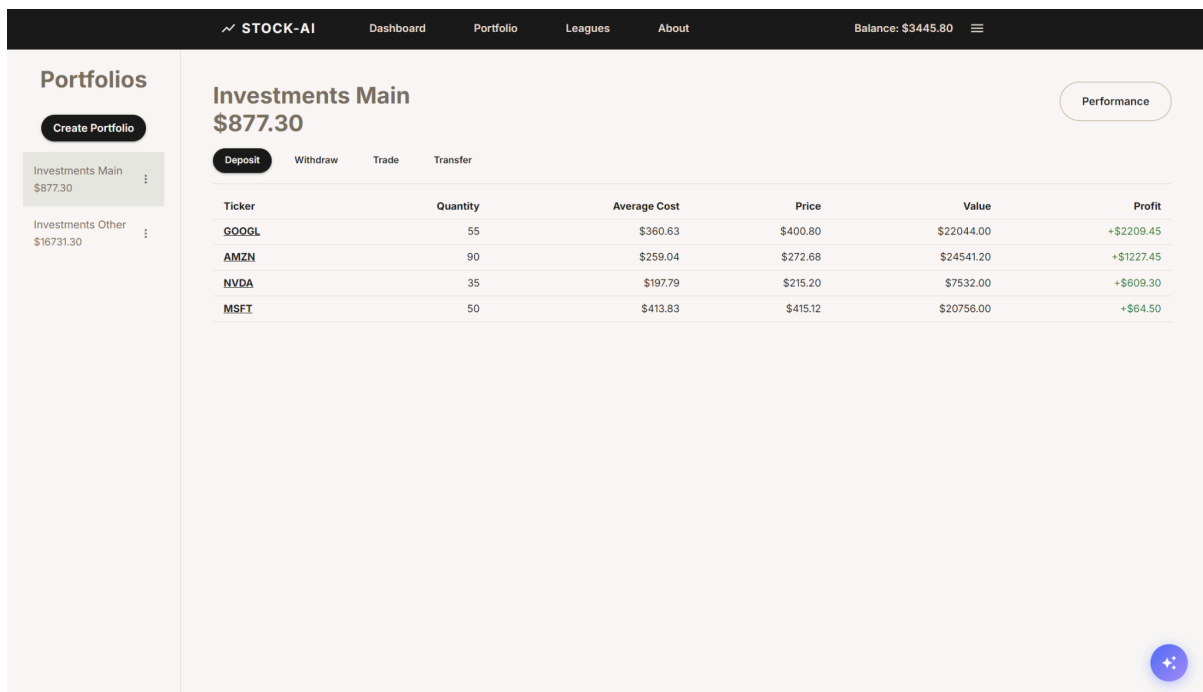


Figure 2: Portfolios view. Note the list of portfolios on the left, with position information (quantity, cost basis, current price, total current value, and net gain/loss) displayed on the right two-thirds of the screen.

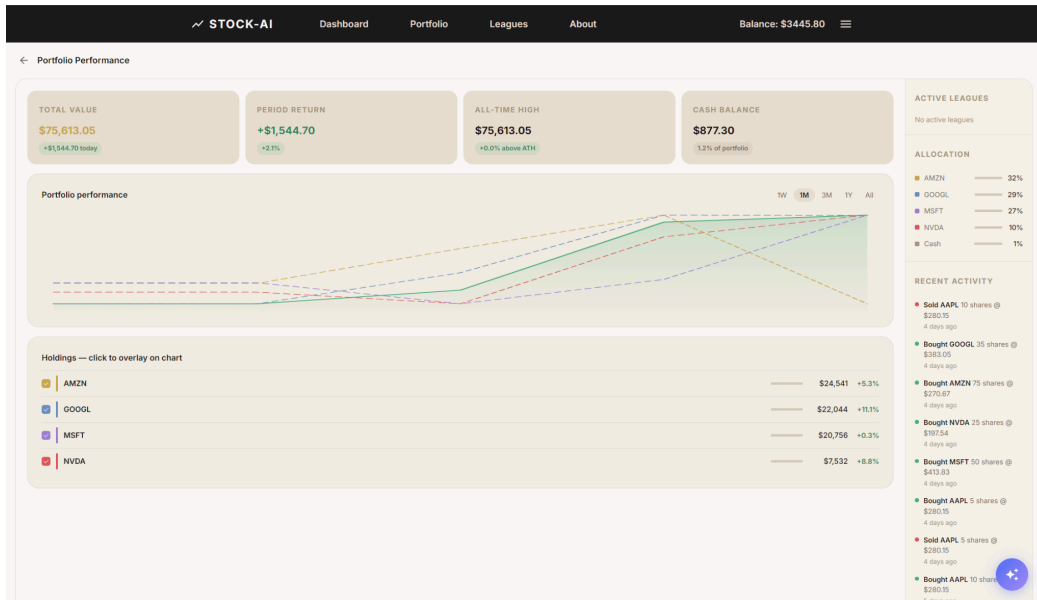


Figure 3: Portfolios performance view. Basic statistics at the top, allocation in the upper right, trade history on the bottom right, and graph view in the middle.

3. Leagues:

Leagues are an optional feature that aims to boost engagement among learners. With it, users can compete against each other by comparing the growth of an entered portfolio. A portfolio is 'snapshot' on entry, and used as a baseline for growth. Comparisons between users are graphed for a visual element. See **Figure 4** below for an example league.

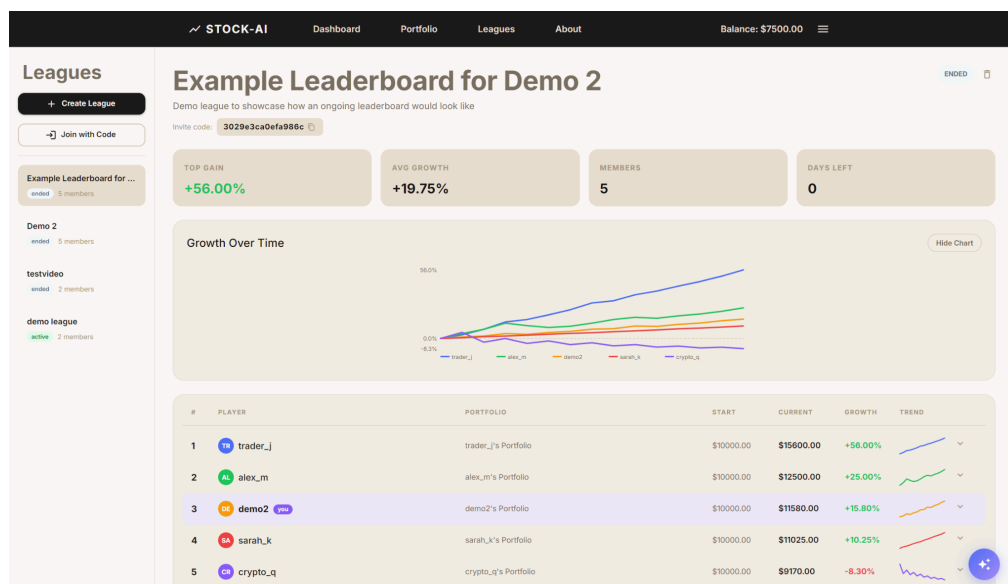


Figure 4: Leagues view, showing an example league with different users' earnings compared.

Design Overview - Tech Stack

Stock-AI follows a fairly standard tech stack for a web application. We are using MariaDB as our database, a FastAPI application to access the database, and a React.js application for our web interface. MariaDB was chosen simply because it was already installed on our server, and it maintains compatibility with standard MySQL. FastAPI was chosen because it offered an easy way to interact with our database and build new endpoints with automatic generation of a Swagger UI documentation page. Similarly, React.js offered an easy way to build out a web UI with different components to be pieced together, which aided the development and implementation of new features. We also utilize other technologies to aid development, such as Podman, Material UI, and Gitlab Runner.

Design Overview - External Resources

For stock data, we are utilizing the yfinance library. It gives Stock-AI both current and historical data to display to the user, and is free to use. It is worth noting that yfinance is not the most constantly updated source of information, due to its nature as a data scraper. This is acceptable for a few reasons; first, Stock-AI is an educational platform, so complete information is more important than down to the second accuracy. Second, Stock-AI is intended to be free, and sourcing more accurate and fully real-time data would incur a non-insignificant cost. Given the breadth of available information through yfinance, it's a worthy tradeoff.

For specific company information such as name, ticker, company logo, description, etc, we are using the FintHub API. It allows us to display a more professional synopsis of a given company, as well as list out news articles that may be relevant for predicting the stock price.

The AI integration in Stock-AI is powered by OpenAI's API. All user prompts/requests are routed through our FastAPI application, which injects context into the prompt to steer the LLM towards what it needs to do - e.g., if a user is asking about AAPL (Apple), the LLM is prompted with "You are a financial advisor specializing in AAPL. The user is asking about AAPL - keep all answers relevant to this stock. Only respond to questions about finance, stocks, and investing. Politely decline any off-topic questions". The result is that the LLM stays on track within the given context.

Implementation - Frontend

Figure 5 pictured below is a diagram of the app's main React components.

AuthContext.tsx wraps the entire app, making authentication available globally.

protectedRoute.tsx prevents the user from navigating past the login and register pages if they are not authenticated. A user becomes authenticated by receiving a JWT from the backend after a successful login. The JWT is included in all subsequent requests sent to the backend. **Figure 6** on the next page demonstrates the flow paths for different users.

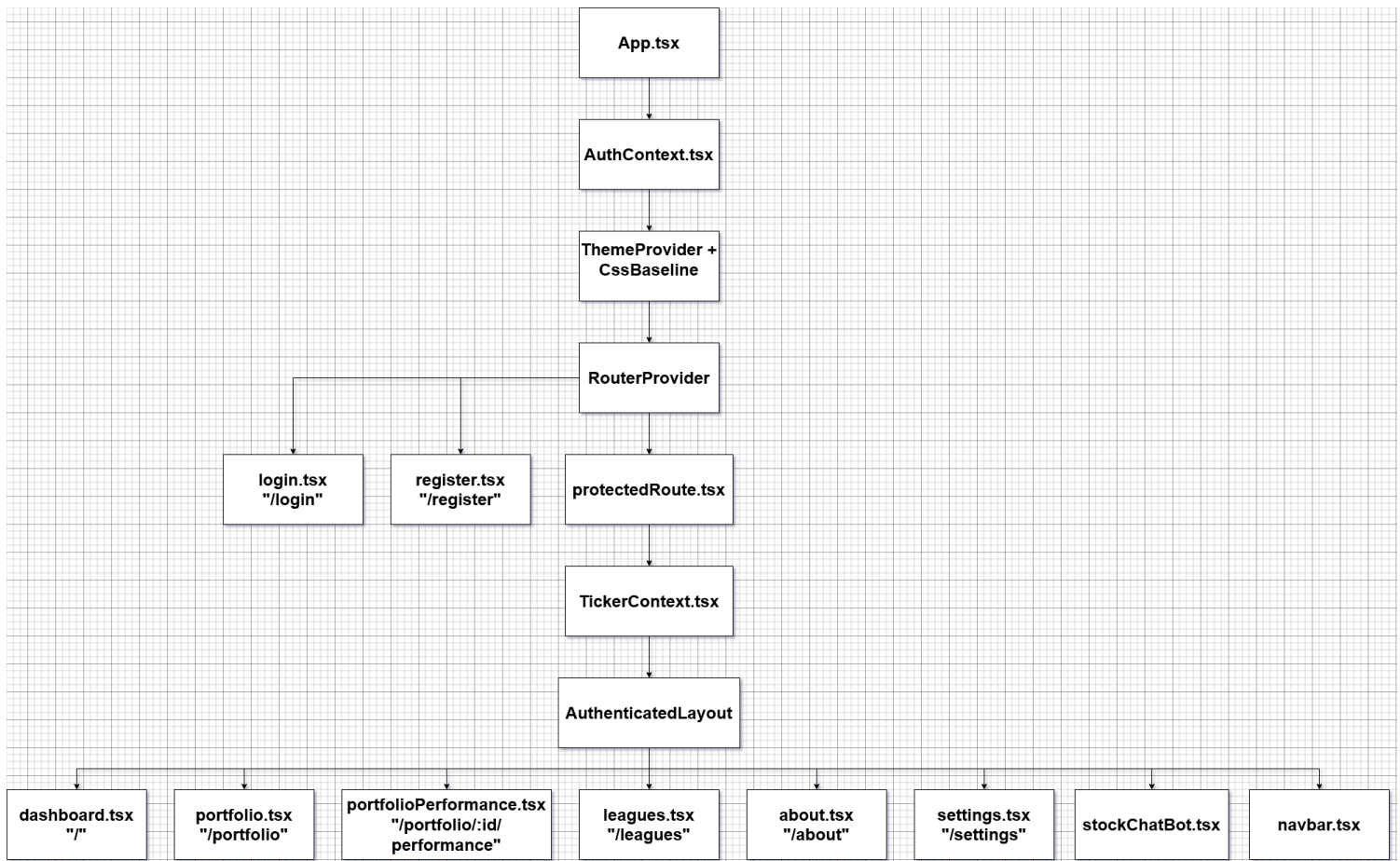


Figure 5: Diagram showcasing the React component structure.

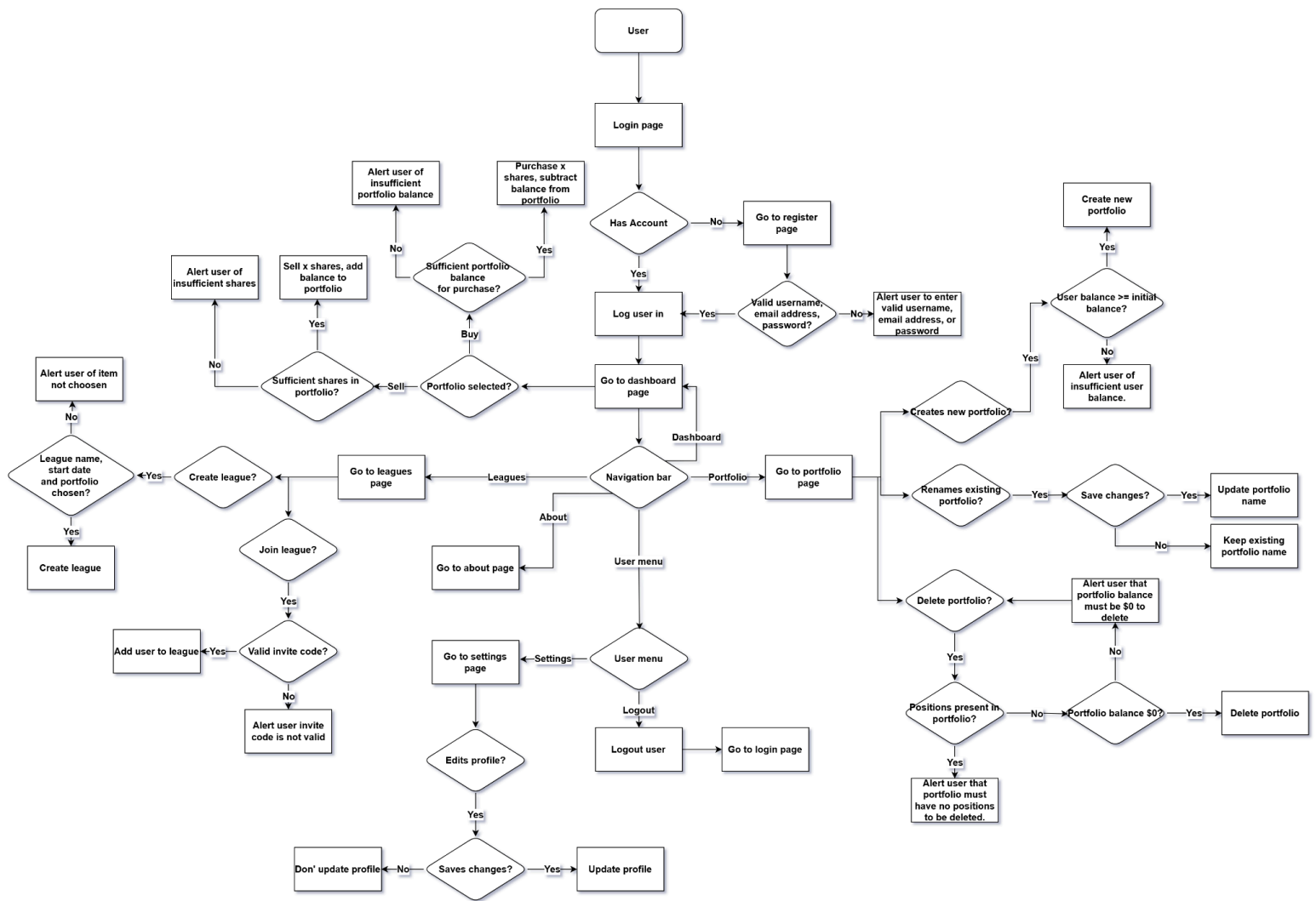


Figure 6: User flow diagram.

Implementation - Backend API

The backend API has many endpoints under multiple categories. For convenience, they are broken down into their respective categories. Category titles serve as the root endpoint for each; i.e., 'get' under 'users' would be '/users/get' if attempting access.

Users

Endpoint	Method	Description
/create	POST	Creates User
/uinfo	GET	Returns basic information about the user.
/login	POST	Log a user in
/protected	GET	Test JWT validity
/update	PATCH	Update user credentials
/delete	DELETE	Soft delete (deactivate) user

Portfolios

Endpoint	Method	Description
/create	POST	Creates new portfolio for user
/rename	PATCH	Renames a portfolio
/portfolio_id/delete	DELETE	Soft delete (deactivate) portfolio
/listall	GET	Lists all of user's portfolios in JSON
/portfolio_id/positions	GET	Lists held positions of portfolio in JSON
/cash_in	PUT	Transfers funds from user to portfolio
/cash_out	PUT	Transfers funds from portfolio to user
/positiontransfer	POST	Transfers a full/partial position from one portfolio to another
/portfolio_id/snapshots	GET	Get portfolio performance snapshots
/portfolio_id/activity	GET	Lists recent trades by portfolio in JSON

Data

Endpoint	Method	Description
/ {ticker} profile	GET	Read stock profile
/ {ticker} /history	GET	Get stock history
/ {ticker} /news	GET	Get stock news

Trades

Endpoint	Method	Description
/newtrade	POST	Performs a mock trade
/ {portfolio_id} /history	GET	Returns list of trades performed under portfolio in JSON

Chat

Endpoint	Method	Description
/conversation/new	POST	Creates a new conversation
/conversation/rename	PATCH	Rename a conversation
/conversation/latest	GET	Gets users most recent conversation, creating one if none exists
/conversation/ {conversation_id}	GET	Gets chat history for a given conversation
/conversation/send	POST	Save user message and return AI reply

AI

Endpoint	Method	Description
/analyze	POST	Trigger AI analysis of a stock

Leagues

Endpoint	Method	Description
/create	POST	Creates a new league
/mine	GET	Lists all leagues the user has entered
/preview/{invite_code}	GET	Preview a league by invite code
/league_id	GET	Get league details
/league_id	DELETE	Delete given league
/league_id/join	POST	Join a league with a chosen portfolio
/league_id/leaderboard	GET	Get ranked leaderboard for given league
/league_id/snapshots	POST	Trigger a snapshot for all members of given league
/league_id/members/{member_id}/snapshots	GET	Get snapshot time-series for a single member

Internal

Endpoint	Method	Description
/snapshots/daily	POST	Daily portfolio snapshot

Implementation - Database

Figure 7 below demonstrates the database tables, their fields, and the relationships that they have with one another.

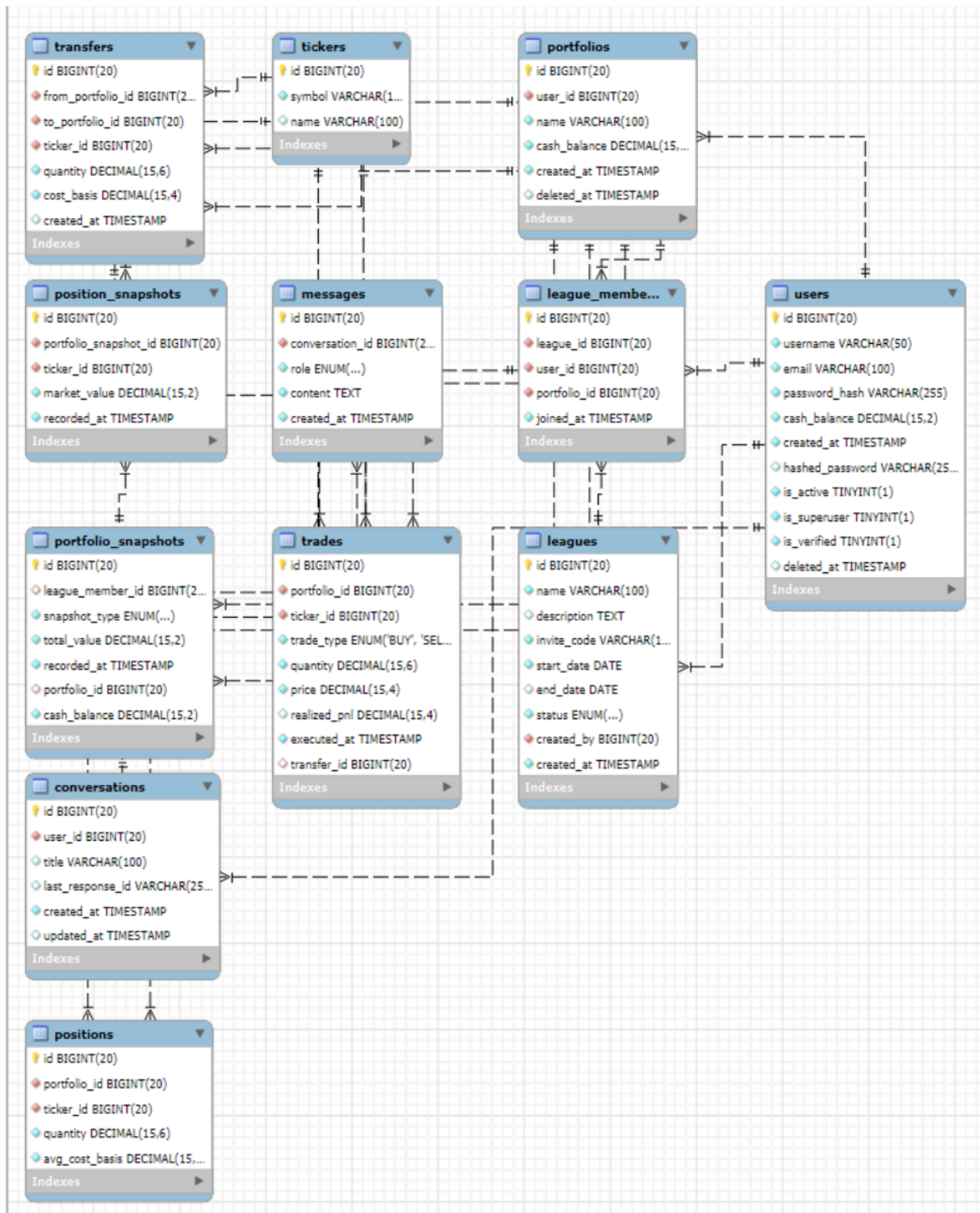


Figure 7: Automatically generated ER diagram via MySQL Workbench.

Work Done

Andrew Butler - Backend development, Client Communication

My first task for the project was to establish basic functionality for the backend - this meant setting up both the MariaDB instance as well as the FastAPI application that would turn into our API. Through the first demo, I had basic user functionality implemented with JWT-based authentication, the beginnings of user portfolios, and a very suboptimal manual deployment using Tmux. The first work for Demo 2 that I did was to learn Docker and Podman to have containerized deployment for the FastAPI application, which enabled us to get CI/CD working so that updates would automatically be pushed to the server. I also flushed out portfolios and trades, enabling mock trading and portfolio management on the backend. I created the initial backend version of conversation history, although it was later modified by Aaron for actual use. My work with demo 3 was lighter; I added the ability for users to update their information and for positions to be transferred between portfolios (sell on donor, buy on destination). I experimented with refactoring our backend to use the more standardized FastAPI Users library for authentication instead of my manually implemented version, but it was incompatible with our existing database relationships, meaning it required a complete rebuild. Throughout the project I also served as the main point of contact with Dr. Phan, scheduling meetings, emailing questions, and communicating progress.

Matthew He - Full Stack development

My contributions to the project spanned both frontend and backend development across all three demos. For Demo 1, I laid out an initial concept and prototype for the stock buying and selling UI, establishing a foundation for how users would interact with trading, though at this stage it was not yet connected to the backend. For Demo 2, I led a redesign of the application's UI to give it a more modern and polished look and feel, helping bring the visual identity of the platform up to the standard the team was aiming for. Also during Demo 2, I implemented the Leagues feature from the ground up on both the frontend and backend, including creating and joining leagues, invite code functionality, portfolio snapshotting on entry to establish a growth baseline, and the leaderboard view that graphs and ranks competitors' portfolio performance over time. For Demo 3, I implemented the Portfolio Performance page, which gives users a detailed view of their portfolio value over time, including a configurable graph with selectable time periods and position overlays, stock allocation breakdown, and a trade history log.

Ethan Roe - Frontend development

My focus for this project was on developing our frontend. My tasks for Demo 1 included creating the first version of our login/signup page and the portfolio page. I worked in tandem with Andrew, who developed the necessary endpoints for those pages in our backend. This also involved creating protected routes for our project, making sure users could not access any pages other than the login and signup pages unless they were logged in. For Demo 2, my main focus was on improving the UI/UX of the portfolios page, since the initial design prioritized functionality over design. I also worked together with Aaron on refactoring our frontend. The refactor mainly focused on breaking up our frontend pages into React components. This made it easier to make changes and add new features to our existing pages moving forward. I also worked on adding the ability to buy and sell stocks within portfolios. For Demo 3, I focused on improving the portfolio's functionality. This included adding the ability to transfer positions between portfolios and the ability to rename and delete portfolios. I also worked on adding a settings menu so users could modify their account details, specifically their username, email, and password.

Aaron Trelstad - Full-Stack Development, AI Integration

My first task for Demo 1 was creating the dashboard, this is the main landing page which enables users to research and analyze different stocks. This involved both frontend and backend work, using the yfinance API for fetching stock related information such as historical prices, recent news, and general company information. For Demo 2, I worked on UI improvements for the dashboard to align it with the theme of the rest of the website and improve user experience. I also started working on how we can integrate AI into our application using the OpenAI API. I created an AI chatbot that took up a static position on the dashboard, it didn't save message history yet. It took in the context of the stock that the user is currently looking at to answer questions related to that specific stock. I also added AI analytics, which gave AI generated signals like BUY/SELL, Risk, Momentum and a Summary. This was designed to help the user get a well rounded perspective of the stock. For Demo 3, I worked on enhancing the AI features, I changed the AI Chatbot to be opened by a button that is present on every screen in the application, this makes the feature more accessible to users. I also added message history which allows a more personalized experience for users. Another improvement was adding explainability to the AI generated analytics, like how/why did it give the BUY/SELL signal or evaluate the current Risk, this helps to give the user more context.

Results

1. Successes:

Stock-AI works well as a free learning platform; it was built with minimal expenses, only requiring sufficient host infrastructure and OpenAI API access. Stock value lookup and historical data is sufficient for an educational focus, with yfinance giving a wide range of available options. Our user portfolios allow flexible organization of positions, as well as useful performance tracking. Leagues also provided a unique take on incentivising learning by provoking a competitive spirit among users. AI analysis serves as a decent indicator for company direction, and the AI chat feature allows persistent conversations that are held on track by our prompt engineering.

2. Could be Improved:

The Stock-AI API, while functional, could use a slight redesign of endpoints to make it more consistent with other applications. Most notably, the inclusion of FastAPI Users or a similar standardized structure. Additionally, the use of the yfinance library as the primary source of stock data introduces inaccuracies, as discussed above. While sufficient for this project at this stage, a more professional deployment would get stock data directly and in real time to gain as much accuracy as possible. Similarly, the non-deterministic nature of AI models could prove confusing for new learners.